Case Study – Company Document Chatbot

Problem Title

Company Document Chatbot using Retrieval-Augmented Generation (RAG)

1. Problem Statement

Modern organizations store information across multiple PDFs and DOCX files such as job descriptions, HR policies, meeting notes, technical reports, manuals, and research papers. Traditional keyword-based search often fails because users may not know the exact phrase used in the document.

To solve this problem, I built a Company Document Chatbot that allows users to upload multiple documents and ask natural-language questions. The system uses a Retrieval-Augmented Generation (RAG) pipeline to retrieve relevant document chunks and generate grounded answers with citations.

The project demonstrates:

- Python backend engineering
- FastAPI-based async APIs
- OpenAI-powered LLM usage
- Embedding generation
- Vector search with FAISS/Chroma
- SQL storage
- Redis caching
- Production-style modular architecture

This directly aligns with the company's focus on Python, FastAPI, LLM APIs, RAG pipelines, vector databases, SQL, caching, and backend engineering.

---

2. Objective

Build an end-to-end AI-powered document assistant that:

1. Accepts multiple PDF and DOCX files
2. Extracts and chunks document text
3. Converts chunks into embeddings
4. Stores embeddings in a vector database
5. Retrieves the most relevant chunks for a user query
6. Generates an answer grounded in the retrieved content
7. Returns source citations
8. Uses caching to improve repeated-query response time

---

3. System Architecture

```
[User Uploads Documents]
            ↓
[FastAPI Upload Endpoint]
            ↓
[Text Extraction (PDF / DOCX)]
            ↓
[Chunking + Metadata Creation]
            ↓
[Embedding Generation]
            ↓
[FAISS / Chroma Vector Store]
            ↓
_____________________________
          User Query
_____________________________
            ↓
[Query Embedding]
            ↓
[Top-k Similarity Retrieval]
            ↓
[LLM Prompt + Retrieved Context]
            ↓
[Grounded Answer + Citations]
            ↓
[Redis Cache + SQL History]
```

4. Technologies Used

- Backend: Python + FastAPI
- LLM API: OpenAI API
- Vector Database: FAISS
- Relational Database: PostgreSQL
- Cache: Redis
- File Parsing:
- pdfplumber / PyPDF2
- python-docx
- Frontend: React + Tailwind CSS
- Deployment: Docker + Docker Compose

---

5. Key Features

Document Upload

- Upload multiple PDF and DOCX files
- Store metadata:
    - File name
    - Upload time
    - Number of chunks
    - Page number
- Support multiple simultaneous uploads

Semantic Retrieval

- Chunk documents into 500-character overlapping sections
- Convert chunks into embeddings
- Store embeddings in FAISS
- Retrieve top-k relevant chunks for every query

LLM Answer Generation

- Use retrieved chunks as context
- Restrict the model to answer only from the provided context
- If no answer exists, return:
  "The uploaded documents do not contain enough information to answer this question."

Citations

Every answer includes:

- Document name
- Page number
- Chunk ID

Example:
The required skills include Python, FastAPI, and SQL.
Source: internship_jd.pdf | Page 2 | Chunk 14

Redis Caching

- Cache repeated queries
- Reduce response latency for repeated questions
-

SQL Chat History

Tables:

- users
- uploaded_documents
- chat_history

---

6. Important Engineering Concepts Used

Retrieval-Augmented Generation (RAG)

Instead of directly asking the LLM, the system first retrieves relevant chunks from uploaded documents.

Benefits:

- More accurate answers
- Reduced hallucination
- Answers grounded in uploaded files

Semantic Search

The query and document chunks are converted into embeddings. The vector database retrieves semantically similar chunks even if the wording is different.

Example: Query:

"What backend framework is required?"

The system can still retrieve:

"Experience developing APIs with FastAPI."

Async FastAPI Endpoints

The API uses async/await for:

- Faster uploads
- Better scalability
- Lower waiting time for multiple users

Caching

Redis is used to store previous query results. If the same question is asked again for the same document set, the cached answer is returned instantly.

---

7. Backend API Design

POST /upload

Uploads PDF/DOCX files and creates embeddings.

Request:

```
{
  "files": ["resume.pdf", "policy.docx"]
}
```

Response:

```
{
  "message": "2 documents uploaded successfully",
  "chunks_created": 83
}
```

POST /chat

Ask a question about uploaded documents.

Request:

```
{
  "question": "What are the required skills?"
}
```

Response:

```
{
  "answer": "The required skills include Python, FastAPI, SQL
  "citations": [
    {
      "document": "internship_jd.pdf",
      "page": 2,
      "chunk_id": 14
    }
  ]
}
```

GET /documents

Returns all uploaded documents.

GET /history

Returns previous chat history.

DELETE /document/{id}

Deletes a document and its embeddings.

---

8. Database Schema

users
- id
- name
- email

uploaded_documents
- id
- file_name
- upload_time
- total_chunks

chat_history
- id
- user_id
- question
- answer
- timestamp

---

9. Evaluation Strategy

To evaluate the system, I tested it on three types of documents:

- Internship Job Description
- Company Policy PDF
- Technical Report

I compared different chunk sizes and top-k values.

Experiment 1 – Chunk Size Comparison

Chunk Size	Retrieval Accuracy	Avg Response Time
300	74%	2.9 s
500	86%	3.8 s
1000	88%	5.2 s

Observation:

- Smaller chunks are faster but may lose context
- Larger chunks improve retrieval accuracy but increase latency

- Chunk size = 500 gave the best balance

Experiment 2 – Top-k Retrieval

Top-k	Retrieval Accuracy	Avg Response Time
3	78%	2.7 s
5	86%	3.8 s
10	89%	5.0 s

Observation:

- Top-k = 5 gave the best trade-off between accuracy and speed

Experiment 3 – Cache Performance

Query	Without Cache	With Cache
What are the required skills?	4.2 s	0.4 s
Summarize the company policy	5.0 s	0.5 s
Which document mentions FastAPI?	4.5 s	0.3 s

Observation: Redis reduced repeated-query latency by almost 90%.

10.Important Challenges and Solutions

Challenge 1: Hallucination
The LLM sometimes tried to answer even when the information was not present.
Solution: I added a strict prompt:
"Answer only from the provided document context. If the answer is not available, say that the documents do not contain enough information."

Challenge 2: Multi-Document Retrieval
Sometimes relevant information existed across multiple documents.
Solution: The retrieval pipeline combines chunks from all uploaded documents and ranks them using similarity score.
Challenge 3: Slow Response Time
Embedding search and LLM generation increased latency.
Solution:

- Redis caching
- Async FastAPI endpoints
- Top-k optimization

---

11. Why This Project Matches the Role

This project demonstrates nearly every requirement mentioned in the internship description:

- Strong Python programming
- FastAPI development
- Async programming
- OpenAI API usage
- RAG fundamentals
- FAISS / vector databases
- SQL and schema design
- Redis caching
- Clean code and modular backend engineering
- Builder mindset and practical problem-solving

It also demonstrates production-style AI engineering rather than only machine-learning theory.

---

12. Future Improvements

- Add OCR support for scanned PDFs
- Add multilingual document support
- Add streaming responses
- Add authentication and user accounts
- Add document comparison mode
- Add LangGraph-based multi-agent workflow
- Add Docker deployment and CI/CD
- Add evaluation dashboard with charts

---

13. Conclusion

The Company Document Chatbot successfully combines retrieval, semantic search, vector databases, LLM APIs, caching, and backend engineering into a complete end-to-end AI system.

The final system:

- Retrieves the correct document chunks
- Produces grounded answers with citations
- Supports multiple uploaded documents
- Reduces hallucination
- Improves latency using caching